

Smart Detour Mechanism in EHPRA

This document explains **how the Smart Detour mechanism works in Enhanced HPRA (EHPRA)** in a clear, step-by-step, and reviewer-friendly manner. The goal is to provide an *explainable* and *implementation-aligned* description that can be used directly as: - a design note, - a supplementary material file, - or the basis of a paper section.

1. Motivation

In conventional detour routing schemes, an intermediate node outside the source–destination bounding box is selected **randomly**. While this approach can help bypass congested or faulty regions, it suffers from two major limitations:

1. The selected detour may still traverse highly congested links.
2. In hotspot traffic patterns, congestion is typically concentrated near the destination, which random detours fail to address effectively.

To overcome these limitations, **EHPRA introduces a Smart Detour mechanism** that replaces random detour selection with a lightweight, congestion-aware decision process.

2. Key Idea of Smart Detour

Instead of selecting a single random detour, Smart Detour evaluates multiple candidate detours and selects the one with the lowest congestion-aware cost.

The mechanism is intentionally designed to remain: - lightweight, - explainable, - and independent of global network state.

Only simple historical link usage counters are used, which are already maintained in the simulator for fault injection and analysis.

3. Available Information

Smart Detour operates using the following information:

- **Mesh topology** (number of rows and columns)
- **Source and destination coordinates**
- **Historical link usage counters** (`link_usage`)

It does **not** require: - queue length information, - global congestion maps, - machine learning models, - or shortest-path recomputation.

This makes the approach suitable for realistic NoC implementations.

4. Step-by-Step Operation

4.1 Bounding Box Construction

First, the minimal bounding box enclosing the source and destination nodes is computed:

```
[min_row, max_row] × [min_col, max_col]
```

This bounding box represents the region covered by minimal routing paths. Detour candidates are selected outside this region.

4.2 Candidate Intermediate Generation

Instead of selecting a single intermediate node, Smart Detour generates **K candidate intermediate nodes** (typically 12–32):

- Each candidate lies outside the bounding box
- Candidates are clamped to mesh boundaries
- Duplicate candidates are discarded

The detour distance is controlled by the EHPRA probability parameter p , ensuring: - small detours when fault/congestion pressure is low, - larger exploration when pressure increases.

This process creates a diverse set of alternative routing options.

4.3 Path Construction for Each Candidate

For each candidate intermediate node `inter`, a two-phase path is constructed:

```
path1 = XY(src → inter)
path2 = XY(inter → dst)
full_path = path1 + path2
```

This preserves the simplicity and deadlock-free properties of deterministic routing primitives.

4.4 Congestion-Aware Cost Function

Each candidate path is evaluated using a cost function that combines:

1) Historical Congestion

For each link (u, v) in the path:

```
congestion += link_usage(u, v)
```

2) Destination-Centric Weighting (Hotspot-Aware)

To handle hotspot traffic effectively, congestion near the destination is weighted more heavily:

```
weight = 1 + C / (distance_to_dst + 1)
```

This reflects the observation that congestion in the final hops toward the destination dominates performance degradation in hotspot scenarios.

3) Hop Penalty

A hop-count penalty prevents excessive path inflation:

```
hop_penalty =  $\lambda$  × hops
```

Final Cost Function

```
score(path) =  $\sum$  [ congestion(edge) × weight(edge) ] +  $\lambda$  × hops
```

The parameters C and λ control the trade-off between congestion avoidance and path length.

4.5 Best Candidate Selection

Among all candidate paths, the one with the **minimum score** is selected as the detour path.

All other candidates are discarded.

This ensures that detouring decisions are: - locally optimal, - congestion-aware, - and dynamically adaptive.

5. Why Smart Detour Works

5.1 Hotlink Traffic

In hotlink scenarios, congestion is concentrated along a narrow corridor. Smart Detour quickly identifies and avoids this corridor by penalizing heavily used links, resulting in significant fault reduction.

5.2 Hotspot Traffic

In hotspot scenarios, congestion accumulates near the destination. Destination-centric weighting ensures that Smart Detour prefers paths that enter the hotspot region from less congested directions rather than simply taking longer detours.

6. Comparison with Conventional Detour Routing

Feature	Conventional Detour	Smart Detour (EHPRA)
Intermediate selection	Random	Multi-candidate evaluation
Congestion awareness	None	Historical link usage
Hotspot awareness	Weak	Strong (dst-weighted)
Complexity	Low	Low-moderate
Explainability	High	High

7. Implementation Characteristics

- Time complexity per routing decision: $O(K \times \text{path_length})$
- Storage requirement: simple link usage counters
- Fully compatible with HPRA probability adaptation logic

Smart Detour therefore enhances routing quality **without sacrificing scalability or simplicity**.

8. Summary

Smart Detour transforms detour routing from a random exploration mechanism into a controlled, congestion-aware decision process. By evaluating multiple candidate detours and incorporating destination-centric congestion weighting, EHPRA achieves superior performance in both hotlink and hotspot traffic patterns while remaining lightweight and explainable.

This mechanism constitutes the primary enhancement of EHPRA over the original HPRA design.